

Programming Assignment-1

Algorithms for Data Science, T2-2026

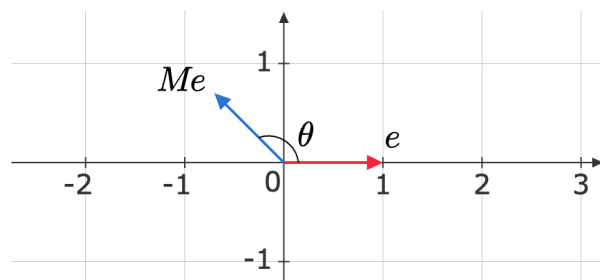
! Instructions

- There are 12 questions distributed across four topics, the first six weeks of the course. This assignment is out of 25 marks.
- You have to submit a report along with the code. We will mainly be evaluating the report and using the code only to verify if you have implemented the solutions. Your report should include all relevant plots and observations gathered from running experiments. Do not add code-blocks in your report.
- AI usage is discouraged. You will learn nothing if you let a chatbot write the all the code and the report. However, we won't penalize you if you use it. We think you are mature enough to make a decision that is good for you in the long-term.
- If your report is not good enough, you will be getting no marks, even if your code is perfect.
- Use this [form](#) to submit your report and code. Only submissions made on this form will be evaluated.
- The deadline is August 9th, 2026, 23:59 PM.

JL Lemma

Question-1: Visualizing random projections (1 mark)

Construct $m = 10,000$ random matrices, $M_1, \dots, M_m$, each of shape 2×2 . For each random matrix, M_i , record the angle (in degrees) between the vectors e and $M_i e$, where $e = (1, 0)$. Plot the histogram of these angles and write down your observation.



$$M = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix}$$

$$m_{ij} \sim \mathcal{N}(0, 1)$$

If you are wondering why we aren't dividing by $\sqrt{2}$ here, we are only interested in the angles and not the distances for this question.

Question-2: JL From Scratch (2 marks)

Consider a dataset X of shape $d \times n$, where $d = 10,000$ and $n = 5,000$ that can be obtained by running the code-block given below.

```
# DATA CELL #
# DO NOT EDIT THIS CELL #
import numpy as np
def get_data(d, n):
    rng = np.random.default_rng(seed = 1234)
    X = rng.uniform(
        size=(d, n)
    )
    return X
d, n = 10_000, 5_000
X = get_data(d, n)
```

Construct an approximate isometry, $f : \mathbb{R}^d \rightarrow \mathbb{R}^k$, and project the dataset to the lower dimensional space, $\mathbb{R}^k$, such that for all pairs of points (x_i, x_j) in the dataset:

$$(1 - \epsilon) \|x_i - x_j\|^2 \leq \|f(x_i) - f(x_j)\|^2 \leq (1 + \epsilon) \|x_i - x_j\|^2$$

where $\epsilon = 0.2$. Ensure that k is significantly less than d .

1. What value of k did you obtain?
2. How many random matrices did you have to generate?

Note that it is not sufficient to just find a suitable k . You have to construct the random matrix and show that it is an ϵ -isometry that satisfies the inequality given above for all pairs of points in the dataset.

Question-3: Effect of k on pair-wise distances (2 marks)

Consider a dataset X of shape (d, n) with $d = 2000$ and $n = 1000$ that can be obtained by running the code-block given below.

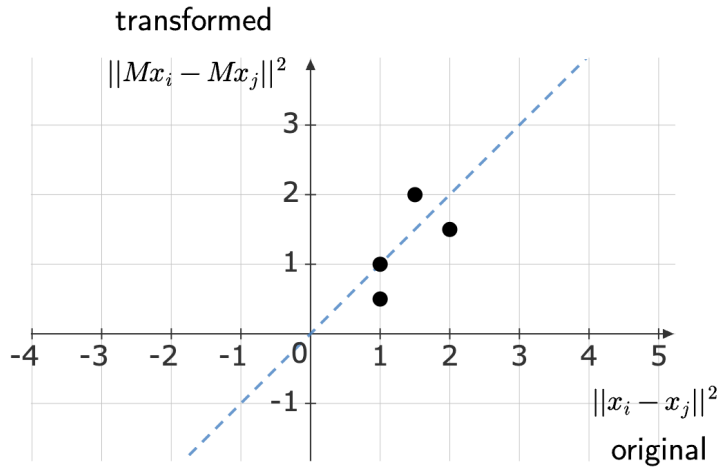
```
# DATA CELL
# DO NOT EDIT THIS CELL
import numpy as np
def get_sparse_data(d, n):
    rng = np.random.default_rng(seed = 1234)
    return rng.choice(
        [0, 1],
        p=[0.8, 0.2],
        size=(d, n)
    )
d, n = 2_000, 1000
X = get_sparse_data(d, n)
```

Construct random projection matrices – as specified by the JL Lemma – that project to $\mathbb{R}^k$ for the following values of k :

- $k = 100$
- $k = 500$
- $k = 1500$

For each of these projection matrices, plot a 2D heatmap of the pairwise distances of the points before and after the transformation. Use **red** to indicate a higher density and **blue** to indicate lower density of points in a region. Compare the three heatmaps and record your observations.

For your reference, the following is a scatter plot of distances for four pairs.



A simple scatter would be too crowded for 1000 points, which is why we want you to present a heatmap.

Question-4: Linear Separability (2 marks)

Run the code given below to initialize a linearly separable dataset (X, y) for a binary classification problem, where the shape of X is (d, n) with $d = 2000$, $n = 1000$ and y is the label vector of shape $(n,)$.

```
# DATA CELL
# DO NOT EDIT THIS CELL
import numpy as np
rng = np.random.default_rng(seed=1035)
d = 2_000
n = 1_000

w = rng.uniform(-2, 2, d)
X = rng.normal(size=(n, d))
y = np.where(X @ w >= 0, 1, 0)
```

- **Verify:** Fit a perceptron to this dataset and verify its linear separability.
- **Experiment:** Construct a random projection matrix – as specified by the JL Lemma – that projects this dataset to $\mathbb{R}^k$. Fit a perceptron on this transformed dataset and measure the accuracy of the resulting model. This constitutes one experiment.

- **Cluster:** Run this experiment for the following values of k : $\{100, 150, \dots, 1000\}$. This set of experiments is one cluster. Run the cluster 50 times and average the accuracies across the 50 runs.
- **Plot:** Plot the accuracies as a function of k and determine the smallest value of k at which the perceptron achieves a 99% accuracy. Record your observations.

Use `sklearn.linear_model.Perceptron` in its default configuration for fitting all perceptrons in your experiments.

LSH

Question-1: Understanding the Datar-hash function (3 marks)

In this question we will verify that the Datar-hash family is (r, c, p_1, p_2) by running simulations. Recall that the Datar-hash family for some w is given by:

$$\mathcal{H}_w = \{h_{ab} : a \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{d \times d}), b \sim U[0, w]\}$$

where, $h_{ab} : \mathbb{R}^d \rightarrow \mathbb{Z}$ is given by:

$$h_{ab}(x) = \left\lfloor \frac{a^T x + b}{w} \right\rfloor$$

The way you will show this is by plotting the collision probability $P(h(x) = h(y))$ as a function of $\|x - y\|$. Refer to the lectures where the professor has drawn a free-hand sketch of this plot. You must get something similar to that. To begin with you can start with points sampled from $\mathbb{R}^2$, that is, $d = 2$. But it is better to set d as a variable that can be changed. Likewise, set $w = 1$ for the first four questions.

Since you have to simulate this whole process, you have to run multiple experiments and average your results across them. Points that we are looking for:

- (1) Set up your experiments and present your results in a manner that shows the dependence of the collision probability on $\|x - y\|$. Specifically, plot the probability of collision of two points x and y as a function of $\|x - y\|$.
- (2) Next, show that the collision probability is independent of the exact values of x and y . For this, you must try out different values of x and y and demonstrate that this choice doesn't affect the final plot.

For both the above problems, your curves must be smooth and not jagged. You have to run multiple trials and average over them to ensure this. For example, each point on the plot should be the average of several trials. Using the plots obtained solve the following:

- (3) Find the value of p_1 and p_2 for $r = 0.5$ and $c = 4$.
- (4) Find the value of r and c for which $p_1 = 0.8$ and $p_2 = 0.1$.

Next, study the effect of w on the collision probability.

- (5) Compare the curves obtained for different values of w . Interpret what you see. Use the expression of the integral obtained in the lectures to bolster your interpretation.

Finally, study the effect of d on the collision probability.

- (6) What effect does d have on the probability curve? Plot the collision probability versus $\|x - y\|$ for values of d in the list $[10, 100, 1000]$. What do you observe? You can get back to $w = 1$ for this question.

Question-2: LSH on a Toy Dataset (3 marks)

Consider a toy-dataset in $\mathbb{R}^4$ generated using the code cell given below. $\mathbf{X}$ is of shape $n \times d$ and $\mathbf{x_query}$ represents the query data-point.

```
# DATA CELL
# DO NOT EDIT THE CELL
import numpy as np
rng = np.random.default_rng(seed=3841)
n = 10_000
d = 4
r = 1
c = 3
X = rng.uniform(-3, 3, size=(n, d))
x_query = rng.uniform(-3, 3, size=(d, ))
```

Use $r = 1$ and $c = 3$ and the Datar-hash family. Any point within a distance r from the query point is a neighbour (positive) and any point that is beyond a distance of cr from the query point is a stranger (negative).

- (1) Find the neighbours and strangers in the dataset. Report the number of neighbours and number of strangers.
- (2) Using the collision-probability plots obtained in **Question-1**, find a good value for w . For this value of w , estimate the values of p_1 and p_2 for the given r and c using the plot.
- (3) Set a suitable value for θ . Using the values that you have estimated so far and the values that you have chosen for the other hyperparameters, find ρ , k and L .
- (4) Construct L hash tables, each of length k , for the query point and the points in the dataset and then check for collisions. Find the number of TP, FP, TN and FN. Compute the precision and recall. Note that precision and recall are with respect to the neighbours (positive) class.

Interpret your observations. Specifically, did LSH solve your problem?

Randomized SVD

Question-1: Randomized SVD from Scratch (3 marks)

We will implement randomized SVD from scratch in this problem using NumPy. Use the following code-block to generate the data-matrix A of shape 400×300 :

```
# DATA CELL
# DO NOT EDIT THIS CELL
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(seed=1934)
from sklearn.datasets import load_sample_image
china_color = load_sample_image("china.jpg")
img = np.dot(china_color[..., :3], [0.2989, 0.5870, 0.1140])
A = img[:400, 50:350]
plt.imshow(A, cmap='gray')
print(A.shape)
```

1. First, compute the SVD of the matrix using the built-in SVD function in NumPy. Find the closest matrix, B , of rank 25 that approximates A . Compute $\|A - B\|_2$ and report your answer. Visualize this approximate matrix as a 400×300 image. Compare this with the original image. What do you observe?
2. Represent the singular values in a log-plot. Use the log-scale for the y-axis, the singular values. For the x-axis, it can be the usual linear scale. Find the index k at which you find an elbow in the graph. The elbow is the point where the graph drops quite steeply and continues to a flat region. Report the value of k and σ_k .
3. Using this value of k just obtained and setting the oversampling parameter $p = 5$, compute the randomized SVD of A . Code up the vanilla algorithm without using power iteration using just NumPy. Let the matrix obtained using randomized SVD be $\hat{A}$. Compute the following quantity.

$$\frac{\|\hat{A} - A\|_2}{\|B - A\|_2}$$

B is the matrix obtained in question-(1).

4. Repeat the above experiment at least 50 times and average over the 50 runs to estimate $\mathbb{E} [\|\hat{A} - A\|_2]$. Now, run this entire setup for five different values of p : $[5, 10, 15, 20, 25]$. Plot the following quantity as a function of p :

$$\frac{\mathbb{E} [\|\widehat{A} - A\|_2]}{\|B - A\|_2}$$

Interpret the plot and explain what is happening.

Question-2: Power-iteration (1 mark)

Generate a random matrix A of shape $(100, 100)$ with each entry drawn independently from a standard normal distribution: $A_{ij} \sim \mathcal{N}(0, 1)$. Compute the SVD of the matrices $(AA^T)^q A$ for $q \in \{0, 1, 2, 3, 4, 5\}$. Plot the six sets of singular values on a common plot with logscale on the y-axis. Describe what you observe and provide an explanation for the same.

Question-3: PCA (2 marks)

Download a dataset of human faces from the web that has at least 5000 images.

- Convert all images to grayscale, resize them to 100×100 and scale them so that the pixel values are in the range $[0, 1]$.
- Center the dataset.
- Run PCA on the centered dataset using randomized SVD as the solver retaining the top 20 components. You can refer to scikit-learn's PCA function.
- Plot σ_i versus i , where σ_i is the i^{th} singular value, for $1 \leq i \leq 20$.
- Visualize the first 20 singular vectors as images.

Some datasets you can use:

- [Human faces, Kaggle](#)
- [Caltech](#)

Note that you don't need to implement randomized SVD from scratch for this problem. Use the implementation provided by scikit-learn.

Spectral Clustering

Question-1: Spectral Clustering using NumPy (3 marks)

Consider the dataset X of 150 points in $\mathbb{R}^3$ generated using the code given below.

```
# DATA CELL
# DO NOT EDIT THIS CELL
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(seed=1001)

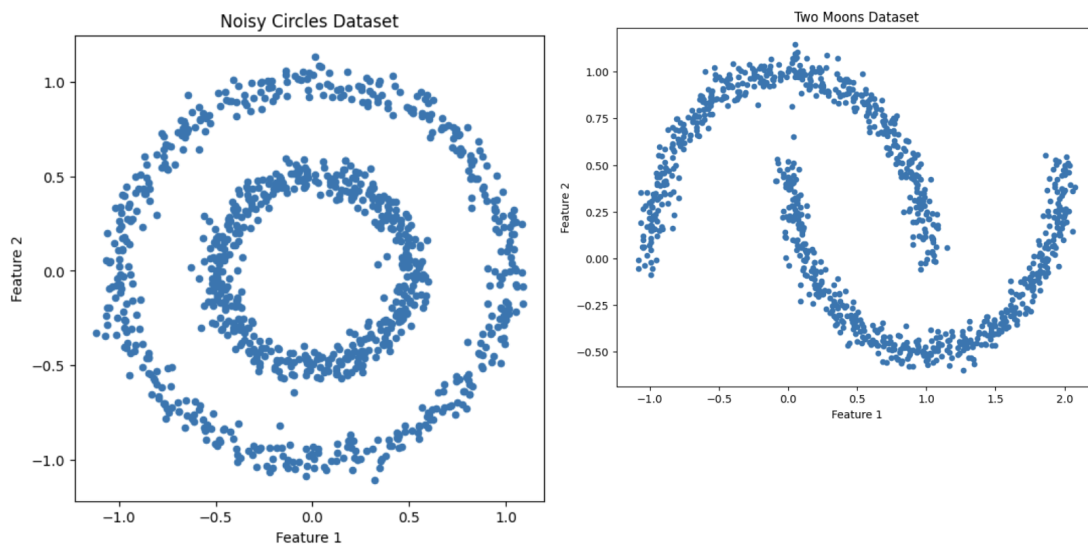
x1 = rng.multivariate_normal(
    mean=[2, 2, 2], cov=np.eye(3), size=50
)
x2 = rng.multivariate_normal(
    mean=[-3, -3, -3], cov=np.eye(3), size=50
)
x3 = rng.multivariate_normal(
    mean=[-2, -2, 6], cov=np.eye(3), size=50
)
X = np.concatenate(
    [x1, x2, x3], axis=0
)
```

Using only the libraries NumPy and Matplotlib, do the following:

- Use the distance metric as $\exp\left(\frac{-||x-y||^2}{2\sigma^2}\right)$ and induce a weighted graph. Use $\sigma = 2$.
- Compute the weight matrix W , the diagonal matrix D and from this the Laplacian L .
- Find the three smallest eigenvalues of the Laplacian.
- Using the eigenvectors corresponding to the two smallest non-zero eigenvalues, construct 2-dimensional embeddings for all the data-points. Ensure that you normalize this embeddings to have unit norm. Visualize these embeddings by means of a scatter plot.
- Run Kmeans with $k = 3$ on this embedding and report your observations.

Question-2: Spectral Clustering on Toy Datasets (2 marks)

- Run both Kmeans and spectral clustering on the following datasets and compare the results. Use $k = 2$ for each dataset-algorithm pair.
- Use the sklearn APIs for both algorithms.
- Don't stop with the default parameters for spectral clustering. Explore the API, tune the hyperparameters and see what that does to the resulting clusters.



Question-3: Image Segmentation (1 mark)

Study the image segmentation literature to understand how spectral clustering can be used to segment images. Get hold of some image on the internet and segment it using spectral clustering. You can use the sklearn APIs here. Comment on the results you obtain.